

UNIVERSITÉ ABDELMALEK ESSAADI
ENSA AL HOCEIMA
Transformation Digitale & Intelligence Artificielle (TDIA)

RAPPORT DE PROJET
Développement Web JEE – Architecture MVC1
Application CRUD – Gestion des Produits
Servlet • JSP • JSTL • Apache Tomcat • Maven

Rédigé par :
Haiti Aya

Encadré par :
M. Mohamed CHERRADI
Formateur JEE

Année Universitaire 2024 – 2025

TABLE DES MATIÈRES

1. Introduction	4
1.1 Contexte et motivation	4
1.2 Objectifs du projet.....	4
2. Description du Projet.....	5
2.1 Présentation générale.....	5
2.2 Fonctionnalités implémentées	5
2.3 Technologies utilisées	5
3. Architecture du Projet.....	6
3.1 Modèle MVC1 – Principes	6
3.2 Architecture 3-Tiers	6
3.3 Structure du projet Maven	6
3.4 Mapping des URL (web.xml).....	7
4. Implémentation – Couche par Couche	8
4.1 Couche Modèle – dao/Produit.java	8
4.2 Couche DAO – Interface et Implémentation.....	8
4.3 Couche Service – Pattern Singleton	9
4.4 Couche Web – Les Servlets	9
4.4.1 ListProduitServlet	9
4.4.2 AddProduitServlet	10
4.4.3 DeleteProduitServlet	10
4.4.4 EditProduitServlet & UpdateProduitServlet.....	10
4.5 Couche Vue – index.jsp.....	11
5. Analyse des Parties Critiques du Code	12
5.1 Le Pattern Singleton dans la Couche Service	12
5.2 Forward vs Redirect — Choix fondamental.....	12
5.3 Formulaire Dynamique dans la JSP.....	13
5.4 Gestion de la Recherche avec Robustesse	13
5.5 Génération automatique des IDs dans ProduitImpl	14
5.6 Le Descripteur de Déploiement web.xml.....	14
6. Étapes d'Implémentation.....	15
Étape 1 — Création du projet Maven dans Eclipse	15
Étape 2 — Création de la couche Modele (package dao)	16
Étape 3 — Création de la couche Service (package services).....	16
Étape 4 — Création des 5 Servlets (package web).....	17
4.1 ListProduitServlet — Afficher / Rechercher	17
4.2 AddProduitServlet — Ajouter	17

4.3 DeleteProduitServlet — Supprimer	17
4.4 EditProduitServlet — Charger pour modification	17
4.5 UpdateProduitServlet — Sauvegarder la modification	17
Étape 5 — Configuration web.xml	18
Étape 6 — Création de la vue index.jsp	18
Étape 7 — Déploiement et Tests sur Tomcat	19
Étape 8 — Versionning Git et GitHub	19
Récapitulatif des étapes	19
7. Bilan et Perspectives.....	21
7.1 Résultats obtenus.....	21
7.2 Difficultés rencontrées	21
7.3 Perspectives d'amélioration	21
7.4 Compétences acquises	22
8. Conclusion	23
Remerciements.....	24

1. Introduction

Dans le cadre du module de développement web JEE (Java Enterprise Edition), ce projet a pour objectif de concevoir et développer une application web dynamique dédiée à la gestion des produits. Cette application met en œuvre les principes fondamentaux de l'architecture MVC1 (Model-View-Controller) couplée à une architecture 3-tiers, en utilisant les technologies standard de l'écosystème JEE.

L'application permet la réalisation de l'ensemble des opérations CRUD (Create, Read, Update, Delete) sur une entité Produit, en combinant des Servlets Java pour le traitement des requêtes HTTP, des pages JSP (Java Server Pages) pour la présentation, et la bibliothèque JSTL (Java Standard Tag Library) pour la logique d'affichage côté vue.

1.1 Contexte et motivation

Le développement d'applications web d'entreprise nécessite une séparation claire des responsabilités afin de garantir la maintenabilité, l'évolutivité et la réutilisabilité du code. Le patron de conception MVC répond à ces exigences en structurant l'application autour de trois composants distincts : le modèle, la vue et le contrôleur.

Ce projet constitue une première mise en pratique concrète de ces principes dans un environnement JEE réel, déployé sur un serveur Apache Tomcat et géré avec l'outil Maven.

1.2 Objectifs du projet

- Développer une application web CRUD complète pour la gestion des produits
- Mettre en œuvre l'architecture MVC1 avec des Servlets dédiées par opération
- Utiliser JSP et JSTL pour la couche de présentation
- Structurer le projet en 3 couches distinctes : DAO, Service, Web
- Déployer l'application sur Apache Tomcat avec Maven

2. Description du Projet

2.1 Présentation générale

Le projet porte sur la réalisation d'une application de gestion des produits. Chaque produit est caractérisé par les attributs suivants :

Attribut	Type	Description
idProduit	Long	Identifiant unique auto-incrémenté
nom	String	Nom commercial du produit
description	String	Description détaillée du produit
prix	Double	Prix unitaire en DH

2.2 Fonctionnalités implémentées

- Affichage de la liste de tous les produits
- Recherche d'un produit par son identifiant
- Ajout d'un nouveau produit via un formulaire HTML
- Modification d'un produit existant (formulaire pré-rempli)
- Suppression d'un produit avec confirmation

2.3 Technologies utilisées

Technologie	Rôle
Java Servlet (JEE)	Traitement des requêtes HTTP — rôle Controller
JSP (Java Server Pages)	Génération dynamique des pages HTML — rôle View
JSTL 1.2	Balises standard pour la logique de présentation
Apache Tomcat 9	Serveur d'application Java EE
Maven	Gestion des dépendances et compilation du projet
Java 11	Langage de programmation principal

3. Architecture du Projet

3.1 Modèle MVC1 – Principes

L'architecture MVC (Model-View-Controller) divise l'application en trois composants indépendants. La variante MVC1 se distingue par le fait que chaque requête HTTP est prise en charge par une Servlet dédiée à une opération spécifique, contrairement à MVC2 où une unique Servlet frontale (Front Controller) dispatche toutes les requêtes.

Composant	Technologie	Rôle
Model	Produit.java	Représente les données — entité métier
View	index.jsp + JSTL	Affichage HTML dynamique des données
Controller	Servlets Java	Traitement des requêtes et navigation

3.2 Architecture 3-Tiers

Le projet adopte une organisation en trois couches distinctes selon le patron architectural 3-tiers, garantissant une séparation claire des responsabilités :

- Couche Présentation (Web) : Servlets + JSP — gère les interactions utilisateur
- Couche Métier (Service) : ProduitMetierImpl — contient la logique applicative
- Couche Données (DAO) : ProduitImpl — accès et manipulation des données

Flux de traitement d'une requête

Navigateur → Servlet (Controller) → Service (Métier) → DAO → [Données en mémoire]
↓
JSP (View) → Navigateur

3.3 Structure du projet Maven

```
GestionDesProduitsMVC1/
├── pom.xml                    ← Gestion des dépendances Maven
├── README.md
└── src/main/
    ├── java/
    │   ├── dao/
    │   │   ├── Produit.java    ← Modèle (entité)
    │   │   ├── ProduitDAO.java ← Interface DAO
    │   │   └── ProduitImpl.java ← Implémentation DAO (en mémoire)
    │   ├── services/
    │   │   ├── ProduitMetier.java ← Interface Service
    │   │   └── ProduitMetierImpl.java ← Implémentation Service (Singleton)
    └── web/
```

```
├── ListProduitServlet.java
├── AddProduitServlet.java
├── DeleteProduitServlet.java
├── EditProduitServlet.java
├── UpdateProduitServlet.java
└── webapp/
    ├── index.jsp           ← Vue principale
    └── WEB-INF/
        └── web.xml         ← Descripteur de déploiement
```

3.4 Mapping des URL (web.xml)

URL	Servlet	Méthode	Action
/listProduits	ListProduitServlet	GET	Lister / Rechercher
/addProduit	AddProduitServlet	POST	Ajouter un produit
/deleteProduit	DeleteProduitServlet	GET	Supprimer un produit
/editProduit	EditProduitServlet	GET	Charger pour modifier
/updateProduit	UpdateProduitServlet	POST	Sauvegarder la modification

4. Implémentation – Couche par Couche

4.1 Couche Modèle – dao/Produit.java

La classe `Produit` représente l'entité centrale de l'application. Elle encapsule les données d'un produit et expose ses propriétés via des accesseurs (getters/setters), conformément aux conventions JavaBean.

```
package dao;

public class Produit {
    private Long    idProduit;
    private String  nom;
    private String  description;
    private Double  prix;

    // Constructeur vide (obligatoire pour JSP/Spring)
    public Produit() {}

    // Constructeur sans ID (utilisé à l'ajout)
    public Produit(String nom, String description, Double prix) {
        this.nom = nom;
        this.description = description;
        this.prix = prix;
    }

    // Getters & Setters ...
}
```

4.2 Couche DAO – Interface et Implémentation

L'interface `ProduitDAO` définit le contrat CRUD sans imposer de technologie de persistance. L'implémentation `ProduitImpl` utilise une simple liste Java en mémoire, ce qui facilite les tests sans base de données.

```
public interface ProduitDAO {
    void addProduit(Produit p);
    void deleteProduit(Long id);
    Produit getProduitById(Long id);
    List<Produit> getAllProduits();
    void updateProduit(Produit p);
}
```

L'implémentation `ProduitImpl` initialise deux produits de démonstration au démarrage via la méthode `init()`, et génère automatiquement les identifiants en incrémentant la taille de la liste.

4.3 Couche Service – Pattern Singleton

La classe `ProduitMetierImpl` applique le patron de conception Singleton pour garantir qu'une seule instance de la couche métier existe durant toute la durée de vie de l'application. Cela permet à toutes les Servlets de partager la même liste de produits.

```
public class ProduitMetierImpl implements ProduitMetier {

    private static ProduitMetierImpl instance;
    private ProduitDAO dao;

    private ProduitMetierImpl() {
        dao = new ProduitImpl();
        ((ProduitImpl) dao).init(); // Précharge les produits
    }

    public static ProduitMetierImpl getInstance() {
        if (instance == null) {
            instance = new ProduitMetierImpl();
        }
        return instance;
    }
    // Délègue toutes les opérations au DAO ...
}
```

4.4 Couche Web – Les Servlets

4.4.1 ListProduitServlet

Cette Servlet gère deux cas : si un paramètre `idProduit` est fourni dans la requête, elle recherche et affiche uniquement ce produit. Sinon, elle retourne la liste complète.

```
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
    String idParam = req.getParameter("idProduit");
    List<Produit> liste = new ArrayList<>();

    if (idParam != null && !idParam.isEmpty()) {
        try {
            Long id = Long.parseLong(idParam);
            Produit p = metier.getProduitById(id);
            if (p != null) liste.add(p);
        } catch (NumberFormatException e) {
            liste = metier.getAllProduits(); // Fallback si ID invalide
        }
    } else {
        liste = metier.getAllProduits();
    }
    req.setAttribute("listeProduits", liste);
    req.getRequestDispatcher("index.jsp").forward(req, resp);
}
```

4.4.2 AddProduitServlet

Reçoit les données du formulaire via une requête POST, crée un objet Produit, le passe à la couche métier, puis forward vers index.jsp avec la liste mise à jour.

```
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
    String nom = req.getParameter("nom");
    String description = req.getParameter("description");
    Double prix = Double.parseDouble(req.getParameter("prix"));

    Produit p = new Produit(nom, description, prix);
    metier.addProduit(p);

    req.setAttribute("listeProduits", metier.getAllProduits());
    req.getRequestDispatcher("index.jsp").forward(req, resp);
}
```

4.4.3 DeleteProduitServlet

Récupère l'identifiant depuis l'URL, supprime le produit via la couche métier, puis effectue une redirection (sendRedirect) vers listProduits pour afficher la liste rafraîchie.

```
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
    Long id = Long.parseLong(req.getParameter("id"));
    metier.deleteProduit(id);
    resp.sendRedirect("listProduits"); // Nouvelle requête HTTP
}
```

4.4.4 EditProduitServlet & UpdateProduitServlet

EditProduitServlet charge le produit à modifier et le passe à la JSP via un attribut produitEdit. La JSP détecte cet attribut et pré-remplit le formulaire. UpdateProduitServlet reçoit le formulaire modifié et met à jour le produit.

```
// EditProduitServlet - doGet
Produit p = metier.getProduitById(id);
req.setAttribute("produitEdit", p);
req.setAttribute("listeProduits", metier.getAllProduits());
req.getRequestDispatcher("index.jsp").forward(req, resp);

// UpdateProduitServlet - doPost
Produit p = new Produit();
p.setIdProduit(Long.parseLong(req.getParameter("idProduit")));
p.setNom(req.getParameter("nom"));
p.setDescription(req.getParameter("description"));
p.setPrix(Double.parseDouble(req.getParameter("prix")));
metier.updateProduit(p);
```

4.5 Couche Vue – index.jsp

La page JSP unique gère à la fois l'affichage de la liste et le formulaire d'ajout/modification. Elle utilise l'expression EL (Expression Language) et la balise JSTL `c:forEach` pour itérer sur la liste de produits.

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<!-- Formulaire dynamique : Ajout ou Modification -->
<form action="${produitEdit != null ? 'updateProduit' : 'addProduit'}"
method="post">
    <input type="hidden" name="idProduit" value="${produitEdit.idProduit}" />
    <input type="text" name="nom" value="${produitEdit.nom}" />
    <input type="submit" value="${produitEdit != null ? 'Modifier' :
'Ajouter'}" />
</form>

<!-- Tableau des produits -->
<c:forEach var="p" items="${listeProduits}">
    <tr>
        <td>${p.idProduit}</td>
        <td>${p.nom}</td>
        <td><a href="editProduit?id=${p.idProduit}">Modifier</a></td>
        <td><a href="deleteProduit?id=${p.idProduit}">Supprimer</a></td>
    </tr>
</c:forEach>
```

5. Analyse des Parties Critiques du Code

Cette section analyse en détail les mécanismes et choix de conception les plus importants du projet, ceux dont la compréhension est essentielle pour maîtriser l'application.

5.1 Le Pattern Singleton dans la Couche Service

Pourquoi c'est critique ?

Sans le Singleton, chaque Servlet créerait sa propre instance de `ProduitMetierImpl` avec sa propre liste de produits. Les données ajoutées via `AddProduitServlet` ne seraient pas visibles dans `ListProduitServlet`. L'application semblerait « vide » à chaque changement de page.

```
private static ProduitMetierImpl instance; // variable de classe (partagée)

public static ProduitMetierImpl getInstance() {
    if (instance == null) { // Première fois seulement
        instance = new ProduitMetierImpl(); // → crée l'instance unique
    }
    return instance; // Toujours la même instance
}

// Dans chaque Servlet :
private static final ProduitMetier metier = ProduitMetierImpl.getInstance();
```

Le mot-clé `static` sur la variable `instance` garantit qu'elle est partagée entre tous les objets de la classe. Le constructeur est privé pour interdire toute instanciation directe depuis l'extérieur.

5.2 Forward vs Redirect — Choix fondamental

Différence capitale

`forward()` : La requête reste la même, les attributs (`setAttribute`) sont conservés. Le navigateur ne change pas d'URL. Utilisé pour afficher `index.jsp` après `add/edit`. `sendRedirect()` : Une nouvelle requête HTTP est créée. Les attributs sont perdus. Le navigateur change d'URL. Utilisé après `delete` pour éviter la re-soumission.

```
// ✓ FORWARD - utilisé après AddProduit, EditProduit, UpdateProduit
req.setAttribute("listeProduits", metier.getAllProduits()); // attribut disponible
req.getRequestDispatcher("index.jsp").forward(req, resp); // même requête

// ✓ REDIRECT - utilisé après DeleteProduit
resp.sendRedirect("listProduits"); // nouvelle requête → pas de re-soumission
// □ Les attributs req.setAttribute() seraient perdus ici
```

5.3 Formulaire Dynamique dans la JSP

La page index.jsp utilise un seul formulaire pour gérer à la fois l'ajout et la modification d'un produit. Ce mécanisme repose sur l'attribut produitEdit placé dans la requête par EditProduitServlet.

```
<!-- Si produitEdit != null → on est en mode MODIFICATION -->
<!-- Si produitEdit == null → on est en mode AJOUT -->

<form action="${produitEdit != null ? 'updateProduit' : 'addProduit'}"
      method="post">

    <!-- Champ caché : transmet l'ID au servlet UpdateProduit -->
    <input type="hidden"
          name="idProduit"
          value="${produitEdit.idProduit}" />
    <!-- Vide si produitEdit == null → ignoré côté AddProduit -->

    <!-- Pré-rempli en mode modification, vide en mode ajout -->
    <input type="text" name="nom"
          value="${produitEdit.nom}" />

    <!-- Bouton adaptatif -->
    <input type="submit"
          value="${produitEdit != null ? 'Modifier' : 'Ajouter'}" />

</form>
```

5.4 Gestion de la Recherche avec Robustesse

La ListProduitServlet implémente une gestion d'erreur importante : si l'utilisateur saisit un ID non numérique dans le formulaire de recherche, une NumberFormatException est levée. Sans gestion, l'application planterait avec une erreur 500.

```
String idParam = req.getParameter("idProduit");

if (idParam != null && !idParam.isEmpty()) {
    try {
        Long id = Long.parseLong(idParam); // ← peut lever
        NumberFormatException
        Produit p = metier.getProduitById(id);
        if (p != null) {
            liste.add(p);
        }
        // Si p == null : liste reste vide → tableau vide affiché
    } catch (NumberFormatException e) {
        // ← Saisie invalide : on affiche tous les produits (comportement
        gracieux)
        liste = metier.getAllProduits();
    }
} else {
    liste = metier.getAllProduits(); // Aucun paramètre : tout afficher
}
```

```
}
```

5.5 Génération automatique des IDs dans ProduitImpl

L'implémentation DAO utilise une stratégie simplifiée pour générer les identifiants : la taille actuelle de la liste + 1. Ce mécanisme fonctionne correctement pour un prototype en mémoire, mais présente une limitation importante en cas de suppression.

```
public void addProduit(Produit p) {
    p.setIdProduit((long) (produits.size() + 1));
    // □ LIMITATION : Si on supprime le produit 2 d'une liste [1,2,3],
    // le prochain produit ajouté aura l'ID 3 (taille=2, 2+1=3)
    // → Collision d'ID possible !
    // En production → utiliser une séquence auto-incrémentée en base de
    données
    produits.add(p);
}
```

Point d'amélioration

Pour une application réelle, remplacer la liste en mémoire par une base de données (MySQL, PostgreSQL) avec une colonne AUTO_INCREMENT ou une séquence JPA. L'interface ProduitDAO facilitera cette évolution sans modifier les couches Service et Web.

5.6 Le Descripteur de Déploiement web.xml

Le fichier web.xml est le point de configuration central de l'application. Il établit la correspondance entre les URLs et les classes Servlet, et définit les fichiers d'accueil. Chaque bloc servlet + servlet-mapping forme un binôme indissociable.

```
<!-- Déclarer la servlet (nom logique → classe Java) -->
<servlet>
    <servlet-name>list</servlet-name>
    <servlet-class>web.ListProduitServlet</servlet-class>
</servlet>

<!-- Mapper une URL au nom logique -->
<servlet-mapping>
    <servlet-name>list</servlet-name> ← doit correspondre exactement
    <url-pattern>/listProduits</url-pattern>
</servlet-mapping>

<!-- Résultat : GET /listProduits → web.ListProduitServlet.doGet() -->
```

6. Étapes d'Implémentation

Cette section décrit le processus complet de mise en place du projet, depuis la création jusqu'au déploiement, en suivant une progression logique couche par couche.

Étape 1 — Création du projet Maven dans Eclipse

Objectif

Créer la structure de base du projet Java EE avec Maven pour gérer automatiquement les dépendances (JSTL, Servlet API).

- 1 File -> New -> Maven Project -> Cocher Create a simple project
- 2 Renseigner : Group Id = ma.ensa | Artifact Id = GestionDesProduitsMVC1 | Packaging = war
- 3 Clic droit sur le projet -> Properties -> Project Facets -> Cocher Dynamic Web Module 3.1 + Java 11
- 4 Ouvrir pom.xml et ajouter les dependances JSTL, Servlet API et JSP API
- 5 Clic droit -> Maven -> Update Project (Alt+F5) pour telecharger les JARs

```
<!-- pom.xml - dependances -->
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>jstl</artifactId>
  <version>1.2</version>
</dependency>
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>javax.servlet-api</artifactId>
  <version>4.0.1</version>
  <scope>provided</scope>
</dependency>
```

Étape 2 — Création de la couche Modele (package dao)

Objectif

Definir l'entite Produit, l'interface ProduitDAO et l'implementation en memoire ProduitImpl.

- 1 Créer le package dao dans src/main/java
- 2 Créer Produit.java avec : idProduit (Long), nom, description (String), prix (Double)
- 3 Ajouter constructeur vide, constructeur sans ID, getters et setters
- 4 Créer l'interface ProduitDAO.java avec les 5 methodes CRUD
- 5 Créer ProduitImpl.java qui implemente ProduitDAO avec une ArrayList
- 6 Ajouter init() pour precharger 2 produits de test au demarrage

```
public void addProduit(Produit p) {
    p.setIdProduit((long) (produits.size() + 1));
    produits.add(p);
}
public void init() {
    addProduit(new Produit("PC 1", "Sony Vaio 1", 7000.0));
    addProduit(new Produit("PC 2", "Sony Vaio 2", 6000.0));
}
```

Étape 3 — Création de la couche Service (package services)

Objectif

Créer la couche intermediaire avec le pattern Singleton pour partager l'etat entre toutes les Servlets.

- 1 Créer le package services dans src/main/java
- 2 Créer l'interface ProduitMetier.java avec les memes 5 methodes que ProduitDAO
- 3 Créer ProduitMetierImpl.java qui implemente ProduitMetier
- 4 Declarer l'attribut static instance et le constructeur private
- 5 Implementer la methode static getInstance() avec if(instance == null)
- 6 Dans le constructeur : instancier ProduitImpl et appeler init()
- 7 Deleger chaque methode metier au DAO via dao.xxxProduit()

Étape 4 — Création des 5 Servlets (package web)

Objectif

Implementer les 5 Servlets Controller de l'architecture MVC1. Chaque Servlet gere une seule operation CRUD.

4.1 ListProduitServlet — Afficher / Rechercher

1	Creer la classe, etendre HttpServlet, declarer le singleton metier
2	Implementer doGet() : lire le parametre idProduit
3	Si present -> getProduitById(); sinon -> getAllProduits()
4	req.setAttribute + forward vers index.jsp

4.2 AddProduitServlet — Ajouter

1	Implementer doPost() : getParameter(nom, description, prix)
2	new Produit(nom, description, prix) -> metier.addProduit(p)
3	setAttribute(listeProduits) + forward index.jsp

4.3 DeleteProduitServlet — Supprimer

1	doGet() : Long.parseLong(getParameter("id"))
2	metier.deleteProduit(id)
3	resp.sendRedirect("listProduits") — REDIRECT obligatoire pour eviter la re-soumission

4.4 EditProduitServlet — Charger pour modification

1	doGet() : getProduitById(id)
2	req.setAttribute("produitEdit", p) — la JSP detecte cet attribut
3	setAttribute(listeProduits) + forward index.jsp

4.5 UpdateProduitServlet — Sauvegarder la modification

1	doPost() : recuperer idProduit (champ cache), nom, description, prix
2	new Produit() -> setters -> metier.updateProduit(p)
3	setAttribute(listeProduits) + forward index.jsp

Étape 5 — Configuration web.xml

Objectif

Declarer toutes les Servlets et leurs URLs dans WEB-INF/web.xml. Tomcat lit ce fichier au démarrage pour router les requetes HTTP.

- 1 Ouvrir src/main/webapp/WEB-INF/web.xml
- 2 Verifier le namespace : xmlns.jsp.org/xml/ns/javaee version 3.1
- 3 Ajouter la welcome-file-list avec listProduits en premier
- 4 Pour chaque Servlet : bloc <servlet> + bloc <servlet-mapping> avec le meme servlet-name
- 5 Sauvegarder (Ctrl+S) -> 0 erreur dans l'onglet Problems d'Eclipse

Étape 6 — Création de la vue index.jsp

Objectif

Creer la page JSP unique (formulaire + tableau) utilisant JSTL et Expression Language (EL).

- 1 Creer index.jsp dans src/main/webapp/
- 2 Ajouter : <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
- 3 Formulaire avec action dynamique : action="\${produitEdit != null ? 'updateProduit' : 'addProduit'}"
- 4 Champ cache : <input type="hidden" name="idProduit" value="\${produitEdit.idProduit}" />
- 5 Champs pre-remplis : value="\${produitEdit.nom}", value="\${produitEdit.prix}"
- 6 Formulaire de recherche : action="listProduits" method="get"
- 7 Tableau avec <c:forEach var="p" items="\${listeProduits}">
- 8 Liens Modifier (editProduit?id=\${p.idProduit}) et Supprimer avec confirm() JavaScript

Étape 7 — Déploiement et Tests sur Tomcat

Objectif

Configurer Tomcat dans Eclipse, deployer l'application et tester toutes les fonctionnalités CRUD.

- 1 Window -> Preferences -> Server -> Runtime Environments -> Add -> Apache Tomcat v9.0
- 2 Clic droit sur le projet -> Run As -> Run on Server -> Sélectionner Tomcat
- 3 Si erreur port 8080 : double-clic Servers -> changer HTTP vers 8081 -> Ctrl+S
- 4 Navigateur : <http://localhost:8080/GestionDesProduitsMVC1/listProduits>
- 5 Tester affichage : 2 produits precharges (PC 1, PC 2) visibles
- 6 Tester ajout : remplir le formulaire -> verifier le nouveau produit dans la liste
- 7 Tester modification : clic Modifier -> verifier le pre-remplissage du formulaire
- 8 Tester suppression : clic Supprimer -> verifier la boite de confirmation JavaScript
- 9 Tester recherche : saisir un ID -> verifier le filtrage

Étape 8 — Versionning Git et GitHub

Objectif

Initialiser un depot Git local et pousser le projet sur GitHub pour la sauvegarde et le suivi des versions.

- 1 Créer un nouveau depot vide sur github.com (sans README)
- 2 Eclipse : clic droit -> Team -> Share Project -> Git -> Create Repository
- 3 Team -> Commit -> ajouter tous les fichiers (++) -> message de commit -> Commit
- 4 Team -> Push Branch -> coller l'URL GitHub du depot
- 5 Saisir nom d'utilisateur + Personal Access Token (pas le mot de passe)
- 6 Preview -> Push -> verifier sur GitHub que tous les fichiers sont presents

```
# Commandes Git Bash (alternative)
git init && git add .
git commit -m "Initial commit - Projet MVC1 Gestion Produits"
git branch -M main
git remote add origin https://github.com/votre-nom/GestionDesProduitsMVC1.git
git push -u origin main
```

Récapitulatif des étapes

#	Étape	Couche	Statut
1	Projet Maven + configuration	Configuration	Realisee
2	Modele : Produit, DAO, ProduitImpl	DAO / Modele	Realisee
3	Service + Singleton	Service / Metier	Realisee
4	5 Servlets CRUD	Web / Controller	Realisee
5	web.xml	Configuration	Realisee
6	index.jsp avec JSTL	View / Presentation	Realisee
7	Deploiement Tomcat + Tests	Deploiement	Realisee
8	Git + GitHub	Versioning	Realisee

7. Bilan et Perspectives

7.1 Résultats obtenus

L'ensemble des fonctionnalités CRUD ont été implémentées et testées avec succès. L'application respecte pleinement l'architecture MVC1 combinée au modèle 3-tiers, avec une séparation claire entre les couches DAO, Service et Web.

Fonctionnalité	Statut	Commentaire
Afficher la liste des produits	✓ Réalisé	JSTL c:forEach
Recherche par identifiant	✓ Réalisé	Gestion NumberFormatException
Ajout d'un produit	✓ Réalisé	Formulaire POST
Modification d'un produit	✓ Réalisé	Formulaire dynamique
Suppression avec confirmation	✓ Réalisé	JavaScript confirm()
Architecture 3-tiers	✓ Réalisé	DAO/Service/Web
Pattern Singleton	✓ Réalisé	Partage état entre Servlets
Déploiement Tomcat/Maven	✓ Réalisé	WAR généré par Maven

7.2 Difficultés rencontrées

- Configuration du web.xml : Les erreurs de validation XML dans Eclipse ont nécessité la mise à jour du namespace vers xmlns.jcp.org
- Conflit de port 8080 : Tomcat ne pouvait pas démarrer car le port était déjà utilisé — résolu en changeant le port vers 8081
- Pattern Singleton : Comprendre pourquoi il est indispensable pour partager l'état entre les Servlets
- Forward vs Redirect : Choisir la bonne méthode pour éviter les problèmes de re-soumission de formulaire

7.3 Perspectives d'amélioration

- Intégration d'une base de données MySQL via JDBC ou JPA/Hibernate pour une persistance réelle
- Migration vers l'architecture MVC2 avec un Front Controller unique (DispatcherServlet)
- Ajout d'une couche de validation des entrées utilisateur (côté serveur)
- Amélioration de l'interface utilisateur avec Bootstrap ou un framework CSS moderne
- Mise en place d'une authentification et gestion des sessions utilisateurs
- Écriture de tests unitaires (JUnit) pour les couches DAO et Service

7.4 Compétences acquises

- Maîtrise du cycle de vie des Servlets et du protocole HTTP
- Utilisation de JSP avec JSTL et Expression Language (EL)
- Application des patrons de conception MVC et Singleton
- Configuration et déploiement sur Apache Tomcat avec Maven
- Structuration d'un projet en couches indépendantes et maintenables

8. Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux du développement web JEE en implémentant une application CRUD complète selon l'architecture MVC1 couplée au modèle 3-tiers. La séparation des responsabilités entre les couches DAO, Service et Web garantit une application modulaire, maintenable et facilement extensible.

La maîtrise des Servlets Java, de JSP avec JSTL, et de la configuration via web.xml constitue un socle solide pour aborder des architectures plus avancées comme Spring MVC ou Jakarta EE. Les choix de conception effectués — notamment le pattern Singleton pour le service et le formulaire dynamique unique pour l'ajout/modification — témoignent d'une réflexion orientée vers la qualité du code.

Les perspectives d'évolution identifiées, notamment l'intégration d'une base de données et la migration vers MVC2, démontrent que l'architecture adoptée est conçue pour évoluer sans remise en cause des fondations. Ce projet constitue ainsi une base solide pour l'apprentissage progressif des technologies JEE avancées.

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

En premier lieu, j'adresse mes sincères remerciements à Monsieur Mohamed CHERRADI, Formateur JEE à l'ENSA AI Hoceima, pour la qualité de son enseignement, sa pédagogie et sa disponibilité tout au long de ce module. Ses explications claires et ses supports de cours détaillés ont été d'une aide précieuse pour comprendre les subtilités de l'architecture JEE.

Je remercie également l'École Nationale des Sciences Appliquées d'AI Hoceima et l'Université Abdelmalek Essaadi pour les infrastructures mises à disposition et la qualité de la formation dispensée dans le cadre du programme Transformation Digitale et Intelligence Artificielle (TDIA).

Enfin, je remercie mes camarades de promotion pour les échanges constructifs et l'entraide qui ont contribué à enrichir ma compréhension des concepts abordés dans ce projet.

Haiti Aya

ENSA AI Hoceima — TDIA
Année Universitaire 2024 – 2025